

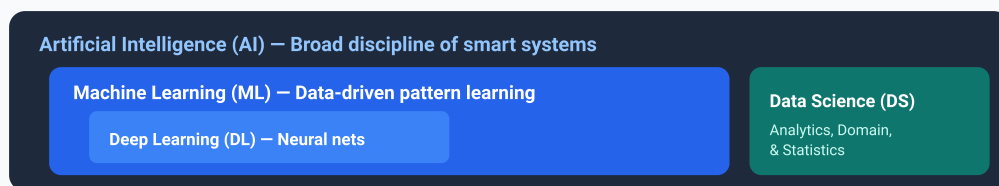
# All Machine Learning Concepts Explained

A concise, high-yield reference guide to foundational AI/ML definitions, taxonomy, learning paradigms, and workflow concepts.

## 01 The AI, ML, Deep Learning & Data Science Hierarchy

Machine learning is a subfield of Artificial Intelligence that focuses on building algorithms that learn patterns from data to make predictions or decisions without explicit rule-based programming.

### AI DOMAIN HIERARCHY & OVERLAP



**Features (Predictors):** Input variables (X) used to make predictions.

**Target / Label (y):** The ground truth output variable to be predicted.

**Observation (Sample):** A single data record containing feature values.

**Structured Data:** Tabular data organized in row/column format.

**Unstructured Data:** Raw media like text, audio, images, and video.

**Parameters vs. Hyperparameters:** Model-learned weights vs. user-set configs.

## 02 The Three Primary Machine Learning Paradigms

### SUPERVISED

#### Supervised Learning

**Input:** Labeled Data (X, y)

**Goal:** Map inputs to known outputs.

**Tasks:** Classification (discrete tags) and Regression (continuous numeric values).

### UNSUPERVISED

#### Unsupervised Learning

**Input:** Unlabeled Data (X)

**Goal:** Discover hidden structures/patterns.

**Tasks:** Clustering, Dimensionality Reduction, Anomaly Detection.

### REINFORCEMENT

#### Reinforcement Learning

**Input:** States, Actions, Rewards

**Goal:** Learn optimal policy via trial & error.

**Key components:** Agent, Environment, Reward signal.

## 03 Supervised vs. Unsupervised Paradigm Breakdown

| Dimension                   | Supervised Learning                                               | Unsupervised Learning                                           |
|-----------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Training Data</b>        | Requires paired inputs with ground truth target labels (X, y).    | Requires input features only without target labels (X).         |
| <b>Primary Objective</b>    | Predict outcomes for unseen data based on learned associations.   | Group similar instances or compress underlying data structures. |
| <b>Common Methods</b>       | Linear/Logistic Regression, Decision Trees, Random Forests, SVMs. | K-Means Clustering, Hierarchical Clustering, PCA, t-SNE.        |
| <b>Real-World Use Cases</b> | Spam detection, medical diagnosis, price                          | Customer segmentation, recommendation                           |

## 04 Essential Machine Learning Algorithms Cheat-Sheet

A quick summary of key classical algorithms covered in the video, categorized by their learning mechanics:

### Linear & Logistic Regression

**Linear Regression:** Fits a straight line (best fit) to predict continuous numerical target variables.

**Logistic Regression:** Applies a Sigmoid activation to output probabilistic classification boundaries (0 to 1).

### Decision Trees & Random Forests

**Decision Tree:** Hierarchical decision rules split on feature thresholds.

**Random Forest:** Ensemble model combining multiple uncorrelated decision trees via *Bagging* (Bootstrap Aggregating).

### Gradient Boosting (XGBoost / LightGBM)

Sequential ensemble method (*Boosting*) where each consecutive tree is trained specifically to correct the residual errors made by preceding trees.

### Support Vector Machines (SVM)

Finds an optimal decision boundary (hyperplane) that maximizes the margin between different classes. Uses kernel tricks for non-linear boundaries.

### K-Nearest Neighbors (KNN)

Instance-based non-parametric algorithm. Classifies new data points based on majority vote among its  $K$  nearest neighbors in vector space.

### K-Means & Principal Component Analysis (PCA)

**K-Means:** Partitions unlabeled data into  $K$  clusters by minimizing centroid distance.

**PCA:** Projects data onto orthogonal axes to reduce dimensionality.

## 05 Data Preprocessing & Feature Engineering Pipeline

Raw data is rarely ready for ML modeling. Preprocessing transforms messy data into clean numerical representations.

### DATA PREPROCESSING SEQUENCE

1. Data Cleaning

2. Encoding

3. Feature Scaling

4. Train-Test Split

### Handling Missing Data & Outliers

- **Imputation:** Filling missing values using Mean, Median, or Mode (or model-based imputation like KNN Imputer).
- **Deletion:** Removing rows/columns when missingness is minimal or extreme ( $> 50\%$ ).
- **Outlier Treatment:** Trimming or capping values exceeding IQR bounds ( $Q1 - 1.5 \cdot IQR$ ,  $Q3 + 1.5 \cdot IQR$ ).

### Categorical Feature Encoding

- **One-Hot Encoding:** Creates binary (0/1) columns for nominal categories without inherent order (e.g., Red, Green, Blue).
- **Label / Ordinal Encoding:** Maps ordered categorical levels into integers (e.g., Low=0, Medium=1, High=2).
- **Target Encoding:** Replaces categories with mean target value.

### Feature Scaling: Normalization vs. Standardization

- **Min-Max Normalization:** Scales values to range  $[0, 1]$ . Best when data distribution is non-Gaussian.

$$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

- **Standardization (Z-Score):** Rescales data to mean = 0 and std dev = 1. Essential for gradient-based models & distance algorithms.

$$z = (X - \mu) / \sigma$$

06 Model Training & Optimization Engine

Training an ML model involves iteratively optimizing parameters to minimize prediction error on training samples.

Train / Validation / Test Split

To evaluate real-world generalization, datasets are partitioned into three subsets:

- **Training Set (~70%):** Used by the algorithm to learn weights.
- **Validation Set (~15%):** Used to tune hyperparameters and prevent overfitting.
- **Test Set (~15%):** Held-out unseen benchmark for final evaluation.

Loss Functions (Cost Functions)

Quantifies the discrepancy between predictions ( $\hat{y}$ ) and true targets ( $y$ ):

- **Mean Squared Error (MSE):** Used in regression; heavily penalizes large errors.

$$MSE = (1/n) \sum (y - \hat{y})^2$$

- **Cross-Entropy Loss:** Used in classification; measures distance between probability distributions.

Gradient Descent & Optimization Dynamics

Gradient Descent is an optimization algorithm that iteratively updates model parameters in the opposite direction of the gradient of the loss function to reach the global minimum.

**Learning Rate ( $\alpha$ ):** Step size taken during each iteration toward a minimum. Too large causes divergence; too small slows convergence.

**Epoch:** One complete forward and backward pass through the entire training dataset.

**Batch Size:** Number of training samples processed before updating parameters (e.g., Mini-batch Gradient Descent).

**Data Leakage:** Critical bug where information from test/validation set leaks into training process.

07 Diagnostics: Overfitting, Underfitting & Bias-Variance

| Condition       | Symptoms                                       | Underlying Cause                                                   | Remedies                                                                            |
|-----------------|------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| UNDERFITTING    | High training error & high testing error.      | Model is too simple (High Bias) to capture data patterns.          | Add features, increase model complexity, reduce regularization.                     |
| OVERFITTING     | Low training error but high testing error.     | Model memorized noise (High Variance) instead of general patterns. | Simplify model, perform feature selection, collect more data, apply regularization. |
| OPTIMAL BALANCE | Low training error & low generalization error. | Balanced trade-off between bias and variance.                      | Cross-validation, early stopping, proper hyperparameter tuning.                     |

Regularization Techniques

Penalizes large weights to restrict overfitting:

- **L1 Regularization (Lasso):** Adds absolute magnitude penalty. Drives coefficients to zero, performing automatic feature selection.
- **L2 Regularization (Ridge):** Adds squared magnitude penalty. Shrinks weights smoothly toward zero.

Early Stopping & Cross-Validation

**Early Stopping:** Halts training as soon as validation loss stops improving.

**K-Fold Cross-Validation:** Splits dataset into  $K$  equal folds; rotates training and validation folds to ensure stable, robust performance metrics.

08 Model Performance Evaluation Metrics

Classification Confusion Matrix

A matrix summarizing performance against actual vs. predicted classes:

|                 | Predicted Positive  | Predicted Negative  |
|-----------------|---------------------|---------------------|
| Actual Positive | True Positive (TP)  | False Negative (FN) |
| Actual Negative | False Positive (FP) | True Negative (TN)  |

Key Metric Formulas

- **Accuracy:**  $(TP + TN) / (TP + TN + FP + FN)$
- **Precision:**  $TP / (TP + FP)$  — Focus on reducing false alarms.
- **Recall (Sensitivity):**  $TP / (TP + FN)$  — Focus on catching all positives.
- **F1-Score:** Harmonic mean of Precision & Recall.

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$
- **ROC-AUC:** Area under ROC Curve (TPR vs. FPR).

**Handling Class Imbalance**

When target classes are unevenly distributed (e.g., 99% non-fraud vs. 1% fraud), Accuracy fails. Solutions include **SMOTE** (Synthetic Minority Over-sampling Technique), random undersampling, or using Precision-Recall Curves.

09 Deep Learning & Neural Network Fundamentals

Deep Learning uses artificial neural networks with multiple hidden layers to automatically extract hierarchical representations from data.

ARTIFICIAL NEURAL NETWORK ARCHITECTURE

Input Layer      Hidden Layers      Output Layer

- Forward Pass: Compute  $\hat{y}$
- Backpropagation: Update  $W$

**Activation Functions**

Introduces non-linearity into networks.

- **ReLU:**  $\max(0, x)$  — standard default.
- **Sigmoid:** Maps to  $(0, 1)$ .
- **Softmax:** Multi-class probability distribution.

**CNNs (Computer Vision)**

**Convolutional Neural Networks** use spatial filters to extract features like edges, textures, and shapes from image grids.

**Transformers & RNNs**

Designed for sequential data (NLP, time series). Modern **Transformers** use self-attention mechanisms to process text in parallel.

